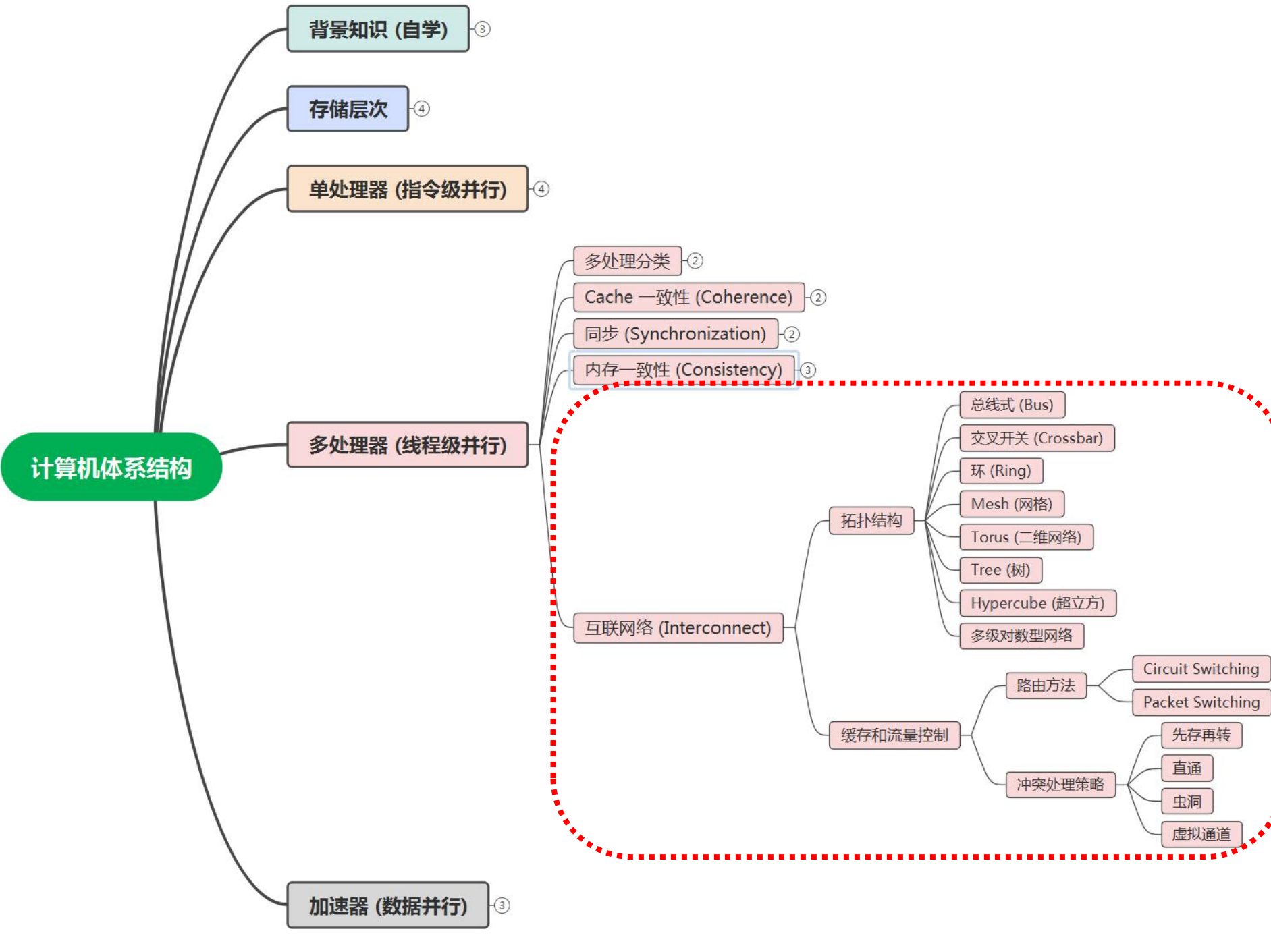
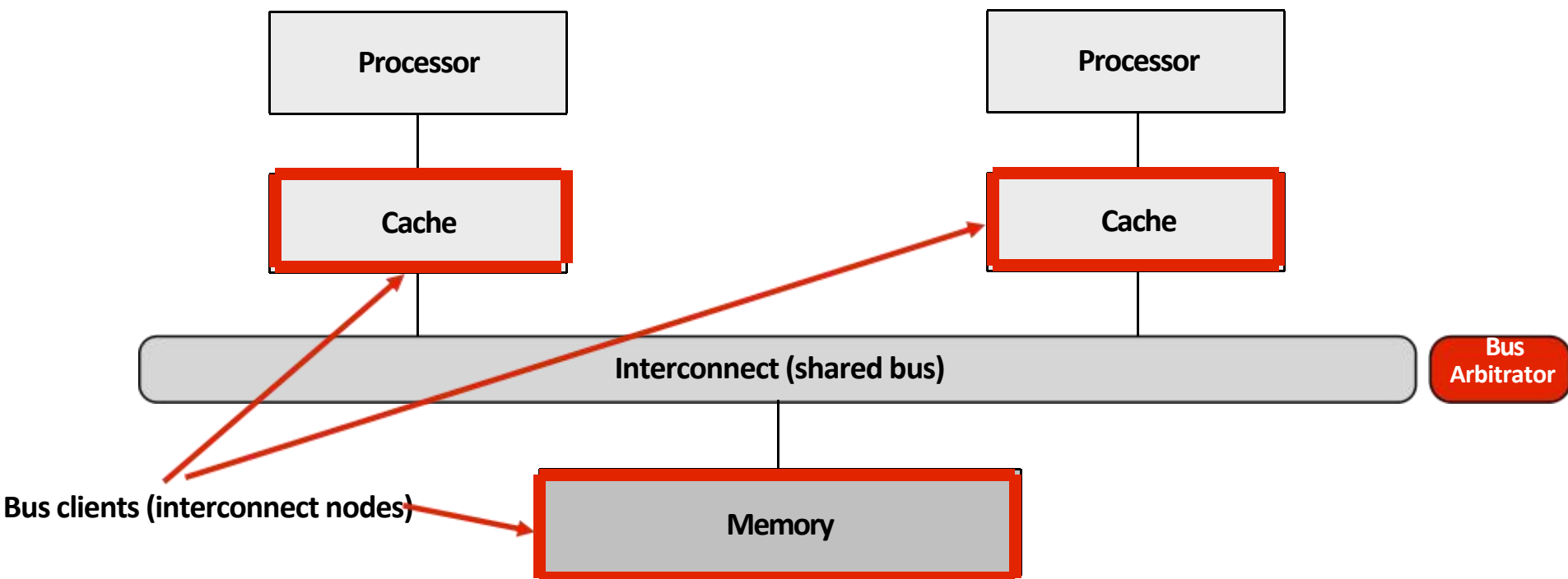


计算机体系结构

L06-5 多处理器互联网络

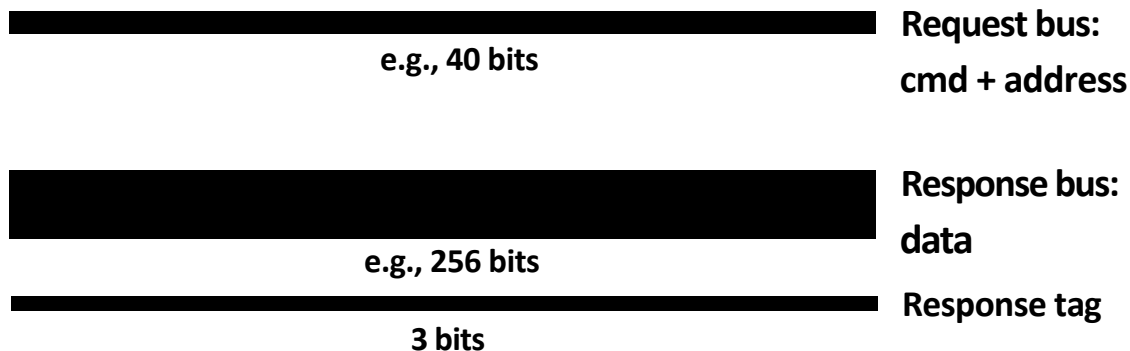


前面讨论的系统基于总线互联

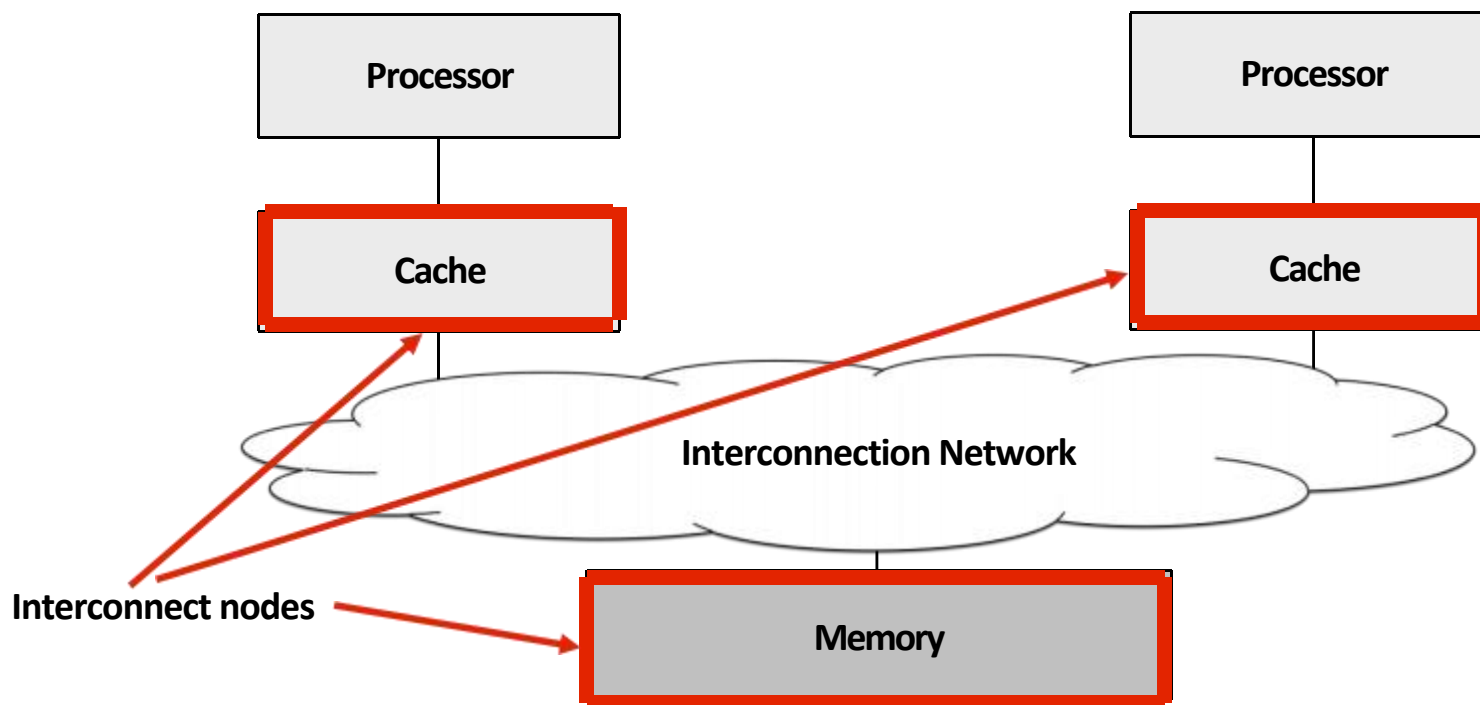


总线互联:

所有节点通过一组共享的导线连接



现代多处理器：互连网络



多个处理器通过片上互连网络连接

为了连接:

- 处理器和处理器
- 处理器和内存
- 处理器和Cache
- Cache和Cache
- I/O设备

互联网络的重要性

□ 系统可扩展性

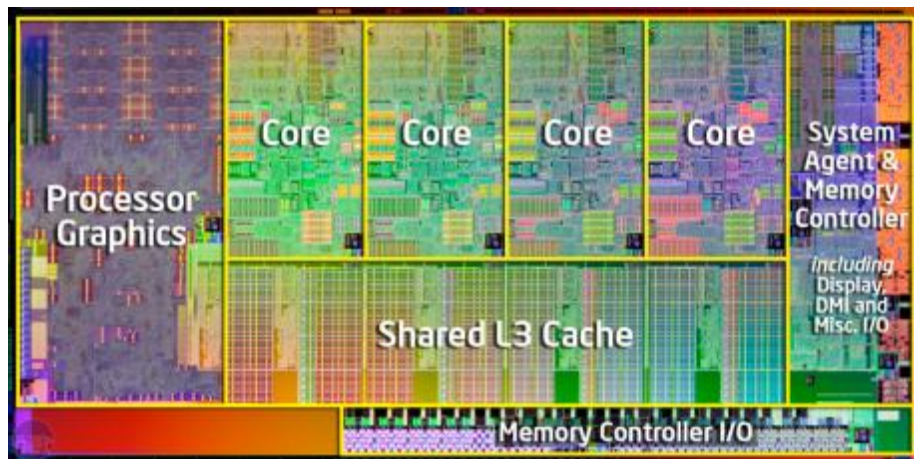
- 系统规模可以多大?
- 增加新节点是否容易

□ 系统性能和功耗

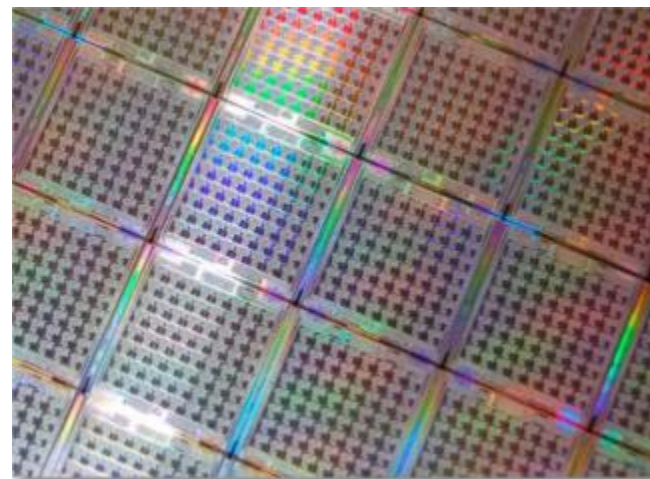
- 处理器, Cache, 内存之间通信的速度有多快?
- 内存访问的延迟有多大?
- 通信的能耗有多少?

随着处理器核数不断增加...

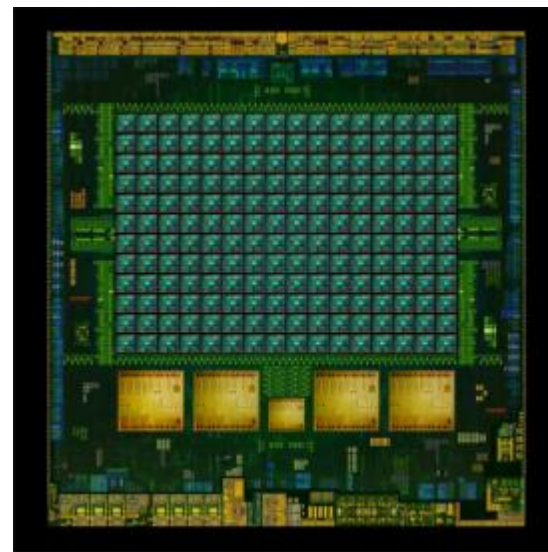
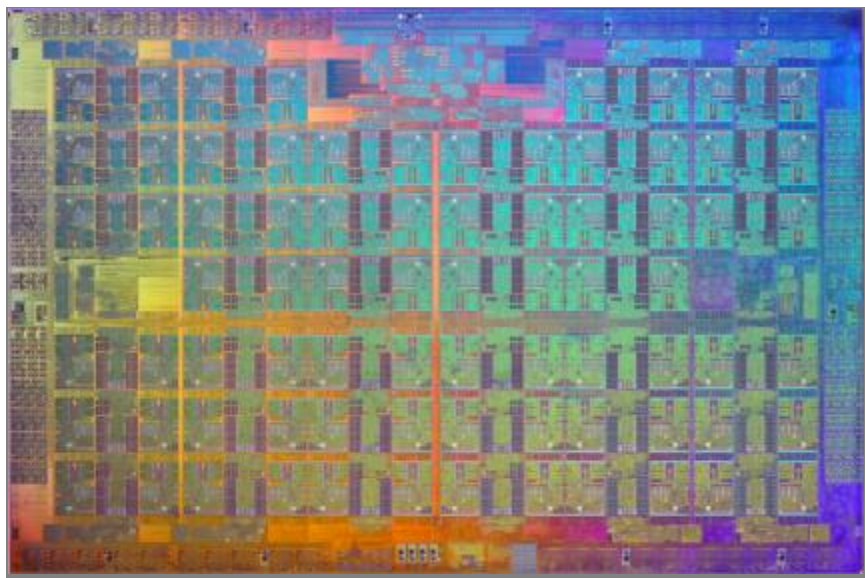
片上网络的可扩展性变得越来越重要



Intel core i7 (4-CPU cores, + GPU)



Tiler GX 64-core chip



Tegra K1: 4 + 1 ARM cores + GPU cores

常用术语

□ Network node(网络节点): 连接到路由器的网络终端节点

- Examples: processor caches, the memory controller

□ Network interface(网络接口):

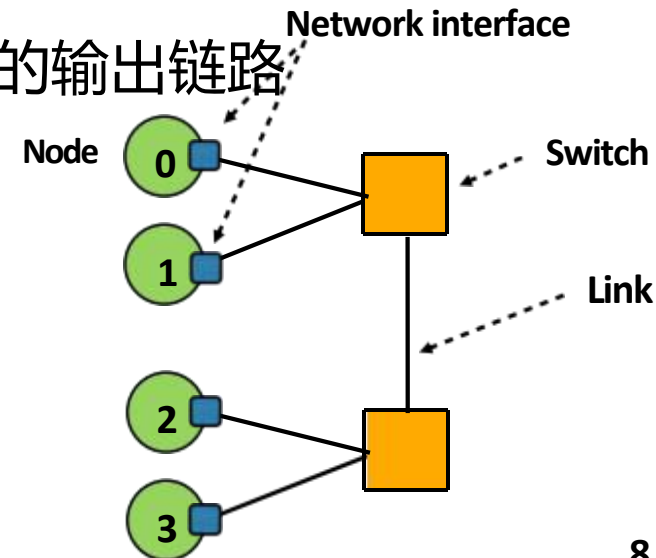
- 将节点连接至网络

□ Switch/router(交换机/路由器):

- 将固定数量的输入链路连接到固定数量的输出链路

□ Link(链路):

- 一束传递信号的导线



设计要点

□ Topology(拓扑结构): 交换机如何通过链路连接

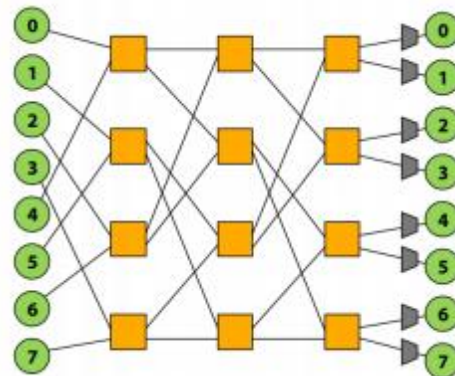
- 影响路由, 吞吐量, 延迟, 实现复杂度

□ Routing(路由): 消息从源节点到目标节点走哪条路径

- 可以是静态决定的,也可以动态决定

□ Buffering and flow control(缓存和流量控制)

- 什么数据存储在网络中? 数据包?部分数据包? etc.
- 网络如何管理缓存空间?



互联网络拓扑结构 (Multi-Issue)

互联网网络拓扑结构的属性

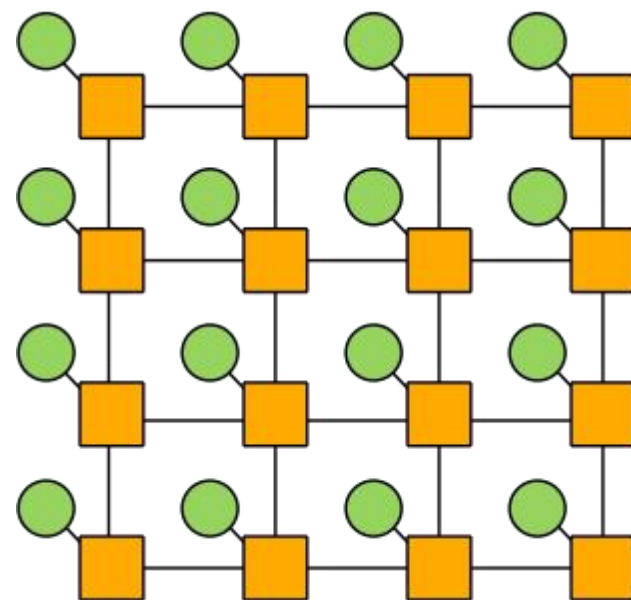
□ Routing distance(路由距离)

- 两个节点之间路由路径的跳数

□ Diameter(直径): 最大路由距离

□ Average distance(平均距离): 平均路由距离

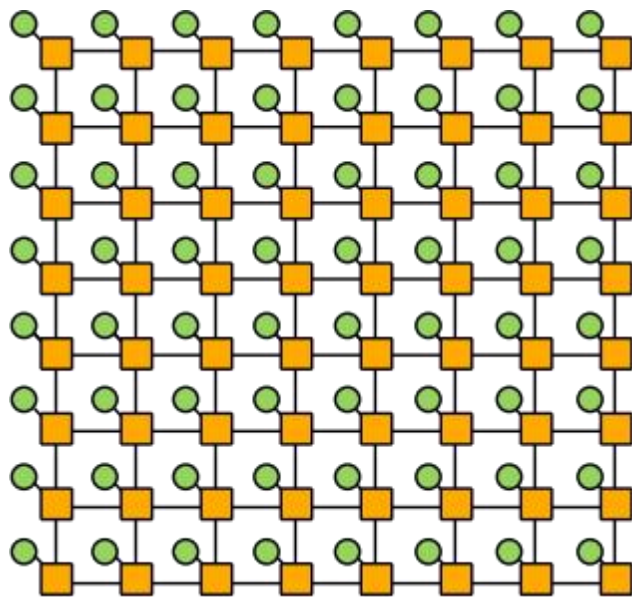
Example:
diameter = 6



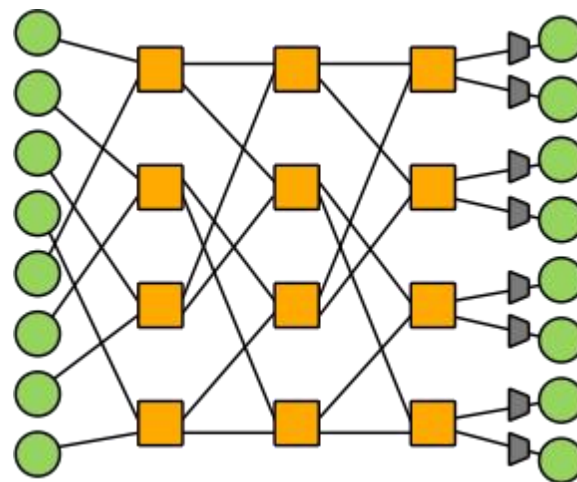
互联网络拓扑结构的属性

□ 直接相连vs. 间接相连

- 直连网络: 终端位于网络内部
- mesh是直连网络: 每个节点既是端点也是交换机



Direct network

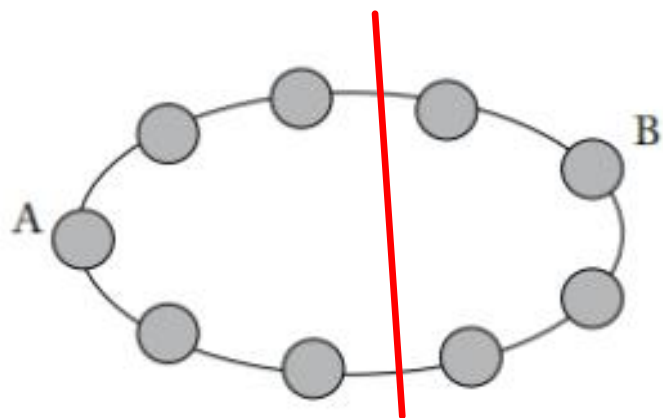


Indirect network

互联网络拓扑结构的属性

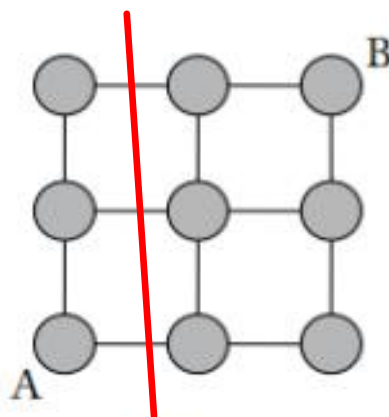
□ 分割带宽(Bisection bandwidth):

- 将网络中的节点分成两个大致相等的部分
- 割带宽是通过这条分割线两侧节点之间的所有网络连接的带宽总和



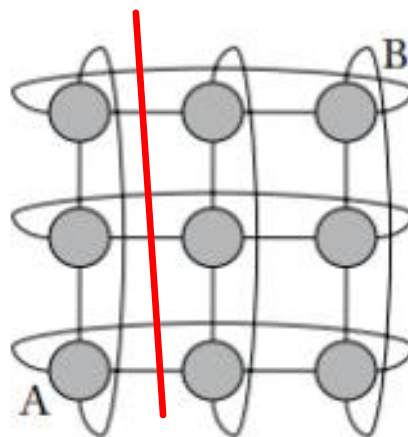
(a) Ring

分割带宽为2



(b) Mesh

分割带宽为3



(c) Torus

分割带宽为6

互联网络拓扑结构的属性

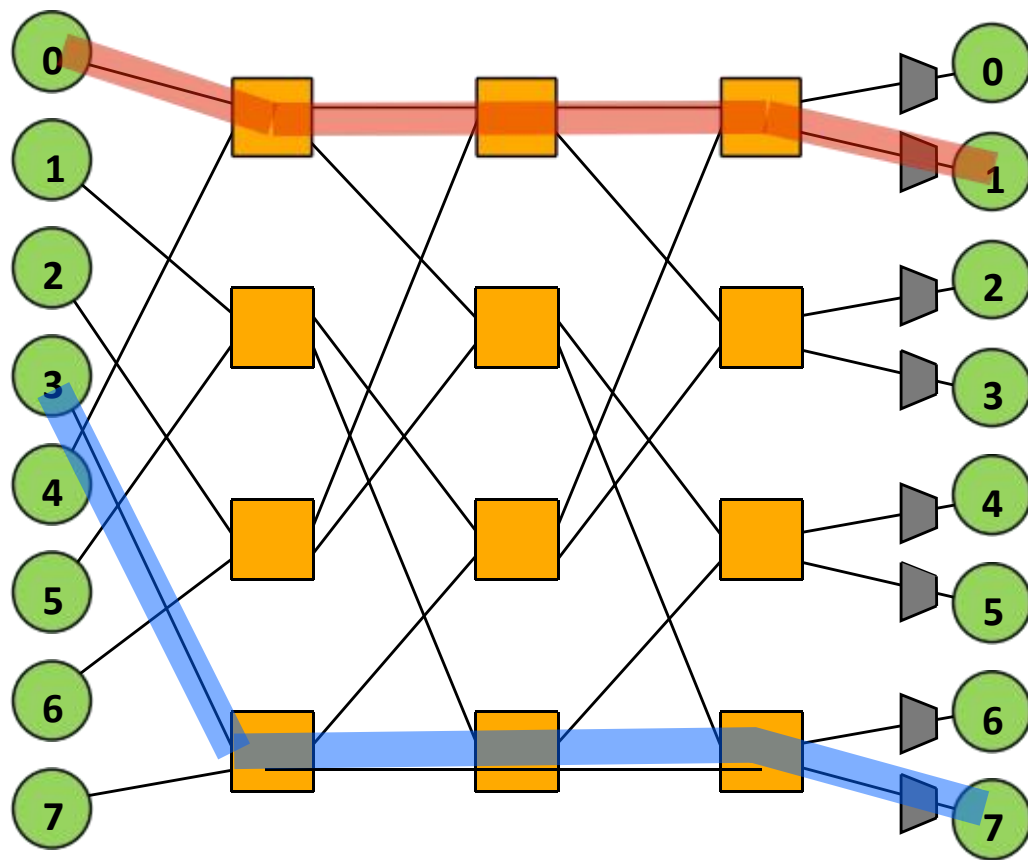
□ 阻塞vs.非阻塞(Blocking vs. non-blocking):

- 在非阻塞网络中，无论有多少数据流同时存在，网络中的数据包都能找到可用路径进行传输，不会产生冲突

Example: blocking vs. non-blocking

□ 该网络是阻塞的还是非阻塞的？

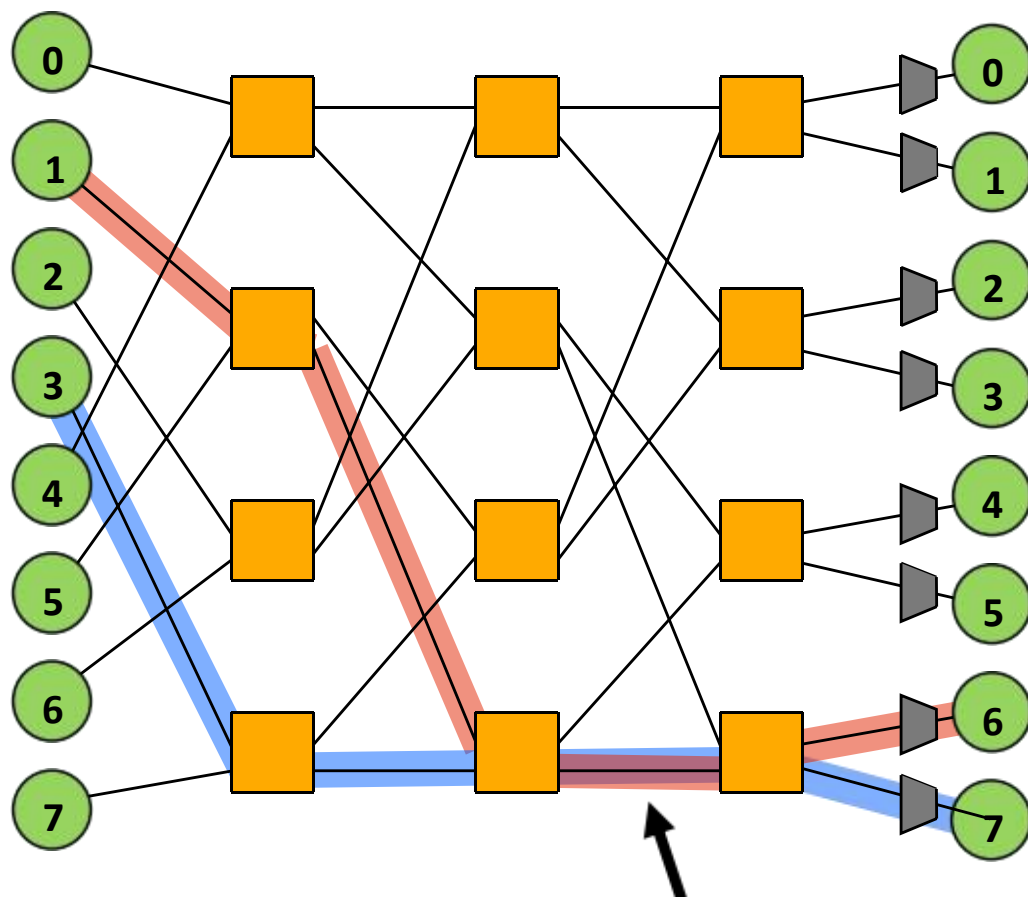
- 考虑从0到1和从3到7的并行消息



Example: blocking vs. non-blocking

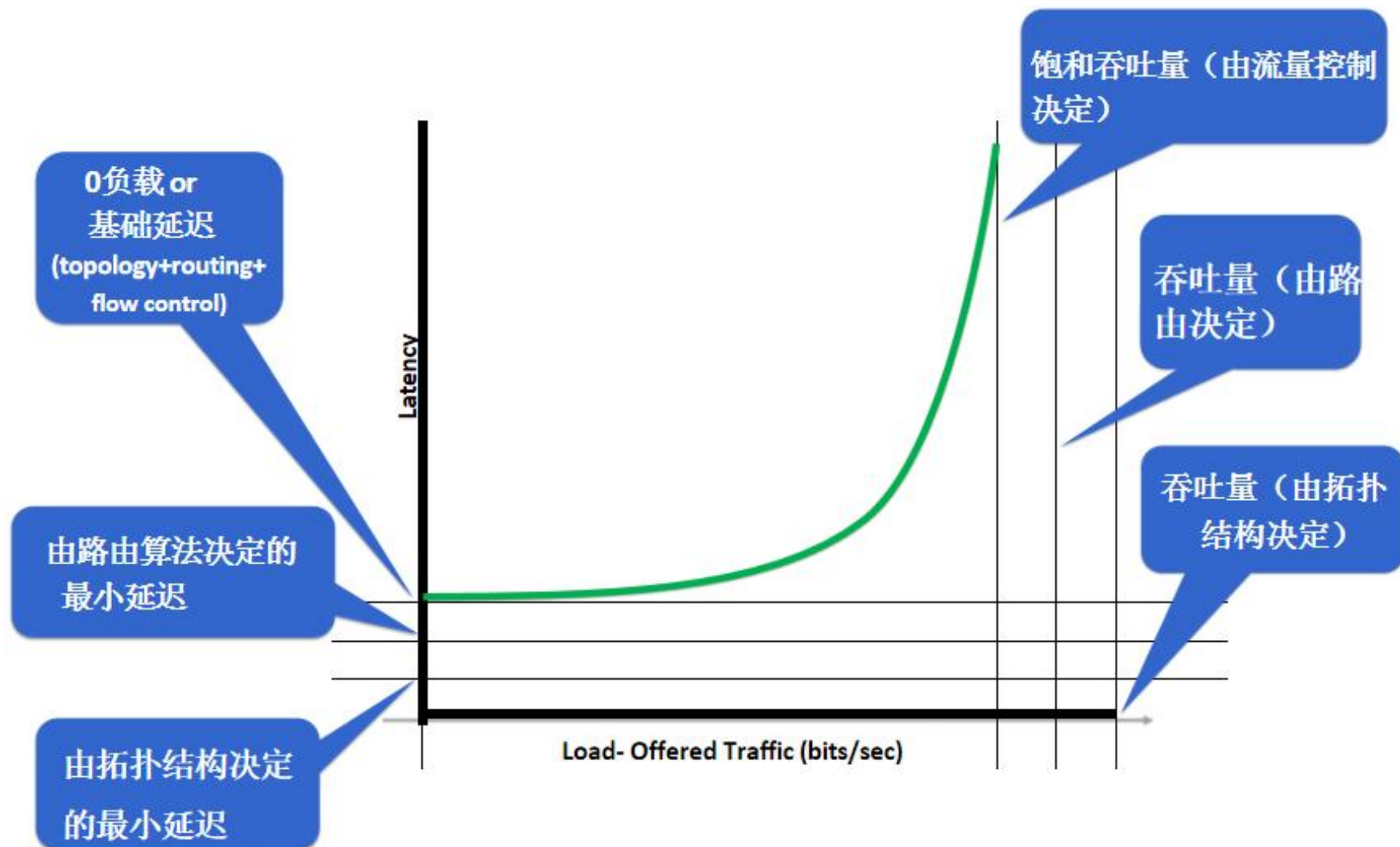
□ 该网络是阻塞的还是非阻塞的?

- 考虑从0到1和从3到7的并行消息
- 考虑从1到6和从3到7的并行消息. Blocking!!!



网络延迟随传输数据量的变化

一般规则：延迟随着负载（吞吐量）的增加而增加



网络拓扑结构

- Bus (总线式)
- Crossbar (交叉开关)
- Ring (环状)
- Tree (树状)
- Hypercube (超立方)
- Mesh (网格)
- Torus (双层网络)
- Butterfly (蝶式)
- ...

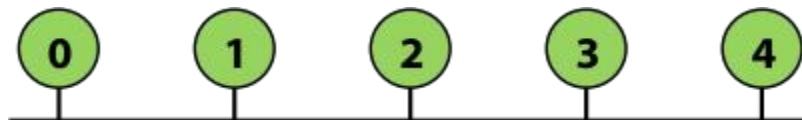
总线式 (Bus)

□ Good:

- 简单
- 规模小时成本低
- 容易实现cache一致性协议(via snooping)

□ Bad:

- 冲突: 所有节点竞争总线
- 带宽有限(所有节点用一条总线)



交叉开关 (Crossbar)

□ 每个节点和其他所有节点互联 (non-blocking, indirect)

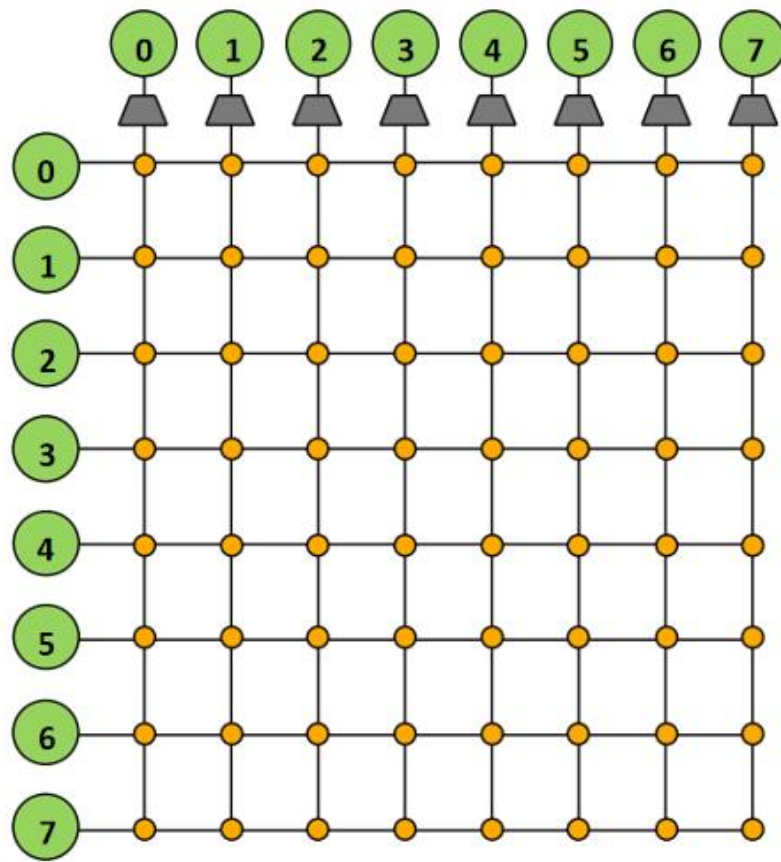
□ **Good:**

- $O(1)$ 延迟和高带宽

□ **Bad:**

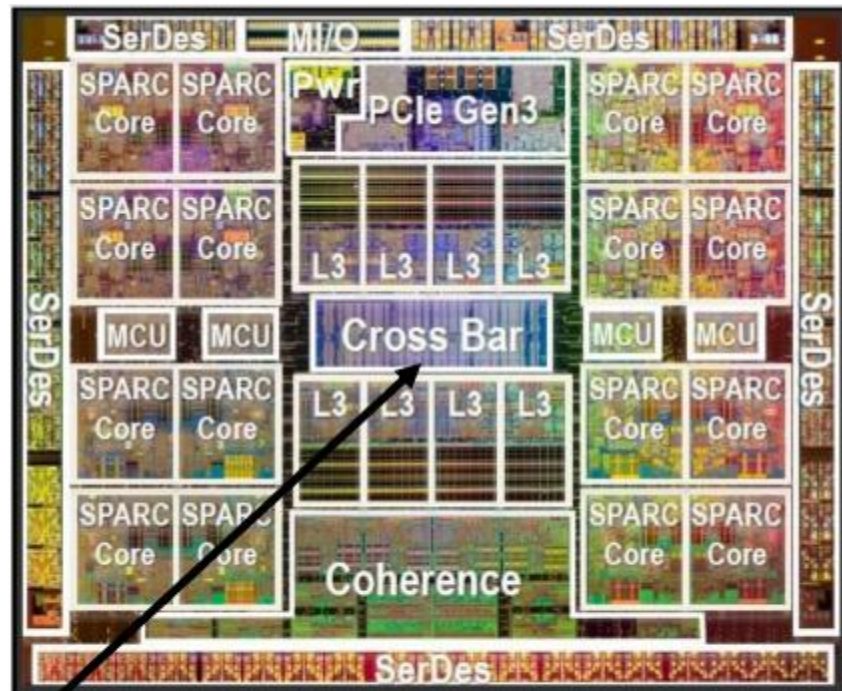
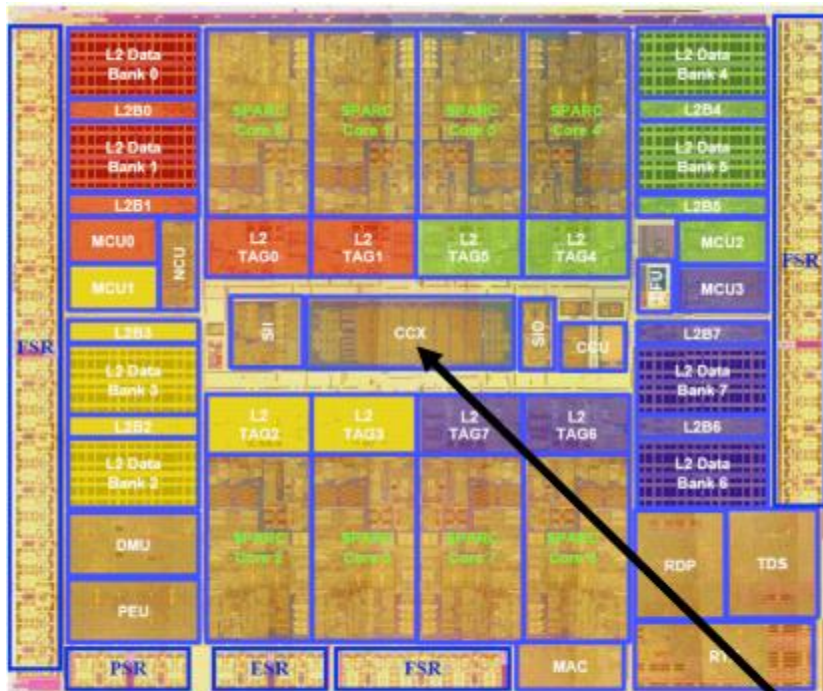
- 扩展性差: $O(N^2)$ switches
- 成本高
- 规模较大时, 仲裁困难

交叉开关调度算法/高效硬件实现
仍然是活跃的研究领域.



8-node crossbar network (N=8)

基于交叉开关的多核处理器



交叉开关所占面积和一个CPU核心差不多

环(Ring)

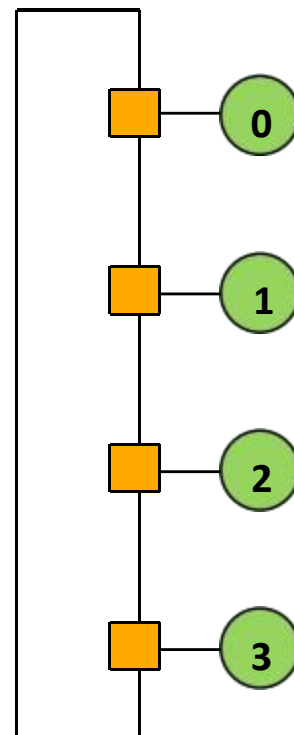
□ Good:

- 简单
- 成本低: $O(N)$ cost

□ Bad:

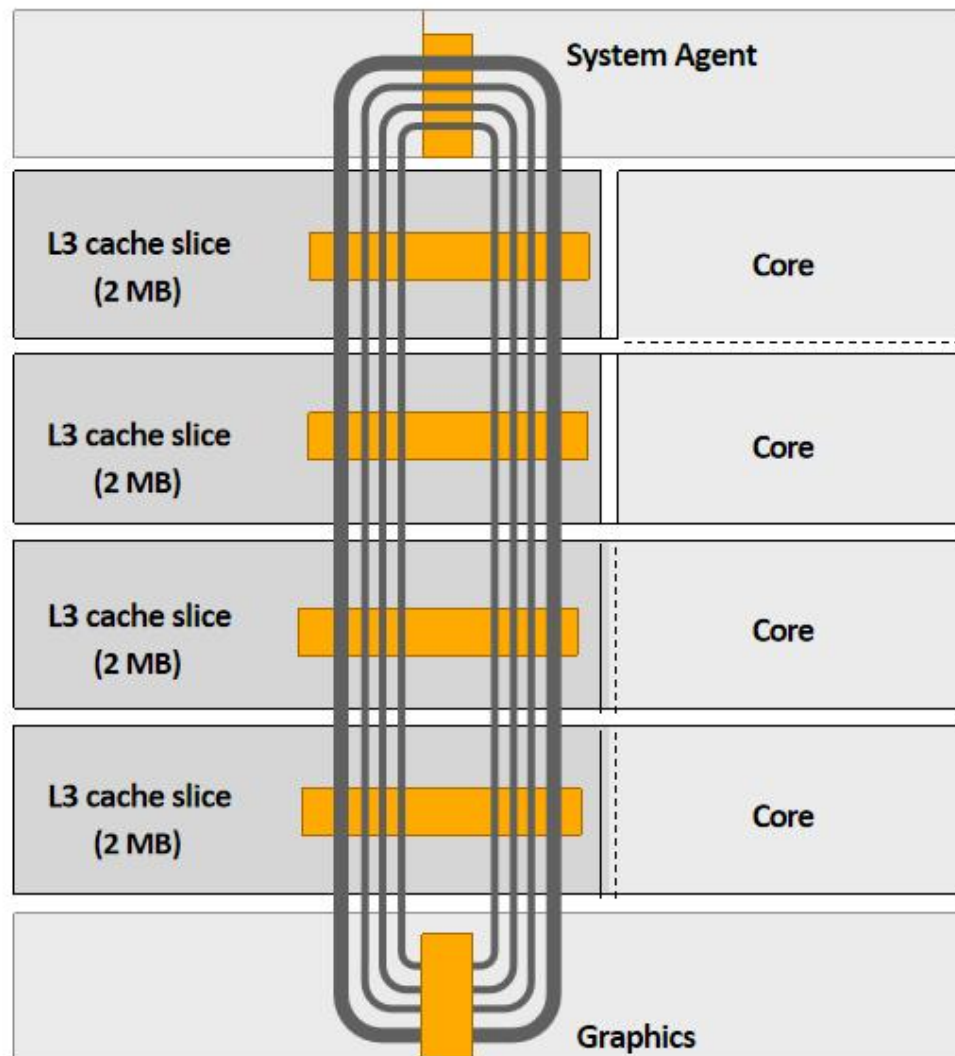
- 延迟高: $O(N)$
- 分割带宽随节点增加保持不变(扩展性差)
- 规模较大时, 仲裁困难

应用在最近的 Intel 架构中 Core i7



Intel' sring interconnect

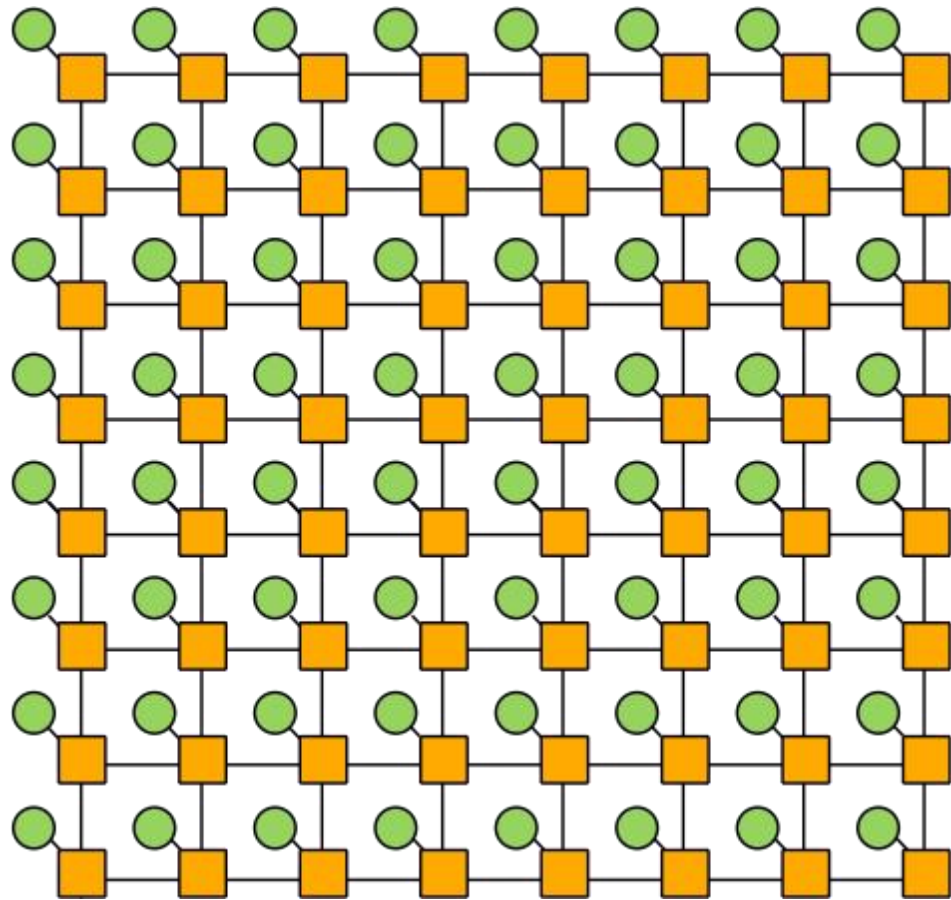
在Sandy Bridge架构中采用



- 四个环
 - request
 - snoop
 - Ack
 - data (32 bytes)
- 六个节点互联
- L3的每个bank连接到环上两次
- CPU->L3理论峰值带宽为 435 GB/sec
 - 当每个core访问本地cache时

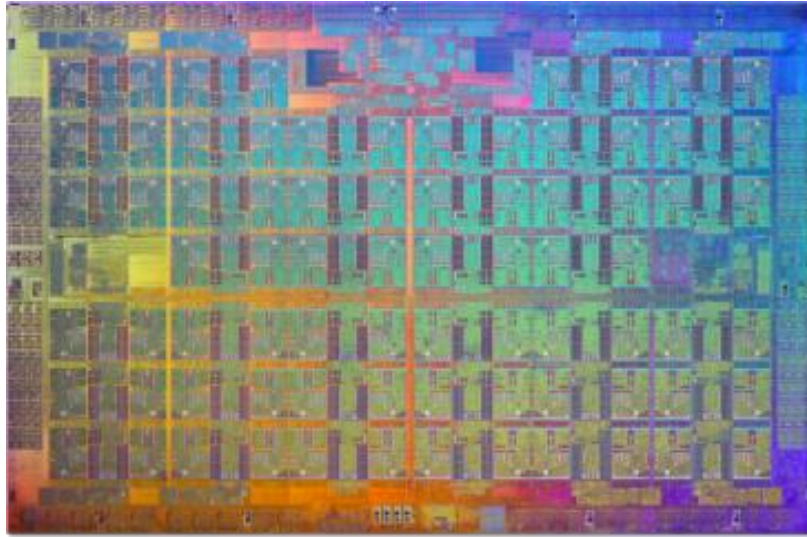
Mesh (网格)

- 直接相连的网络
- 适合具有网格特征的应用
- $O(N)$ 成本
- 平均延迟: $O(\sqrt{N})$
- 容易在芯片上布局: 固定线长
- 可选路径多

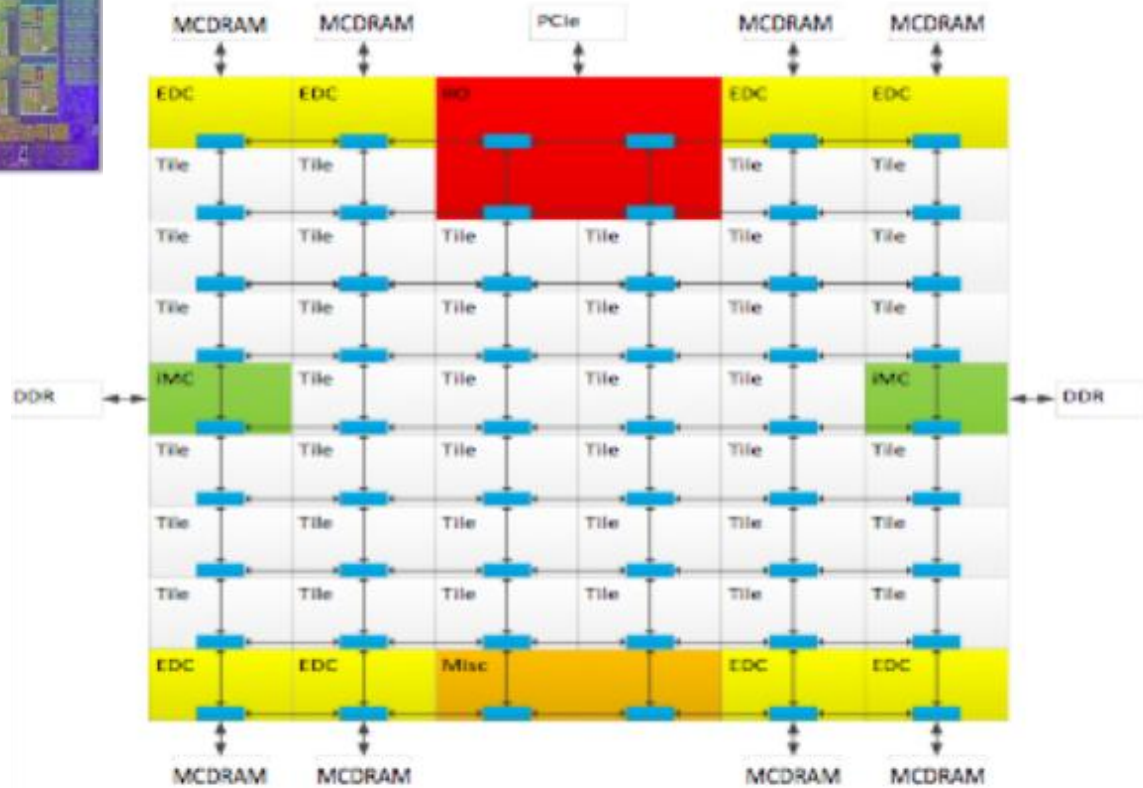


2D Mesh

Xeon Phi (Knights Landing)

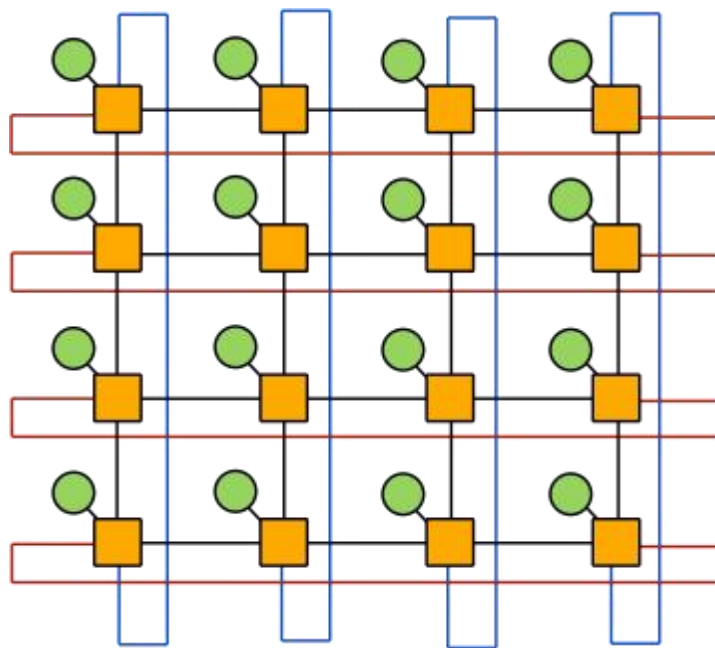


72 cores, arranged as 6 x 6 mesh of tiles (2 cores/tile)



Torus (二维网络)

- 网格拓扑的特性取决于节点位于网络的边缘还是中间
- 环形拓扑引入了新链接以避免这个问题
- 成本仍是 $O(N)$, 但比2D网格高
- 比mesh的路径选择更灵活, 分割带宽更高
- 复杂度更高
 - 片上不好布局
 - 链路长度不相等

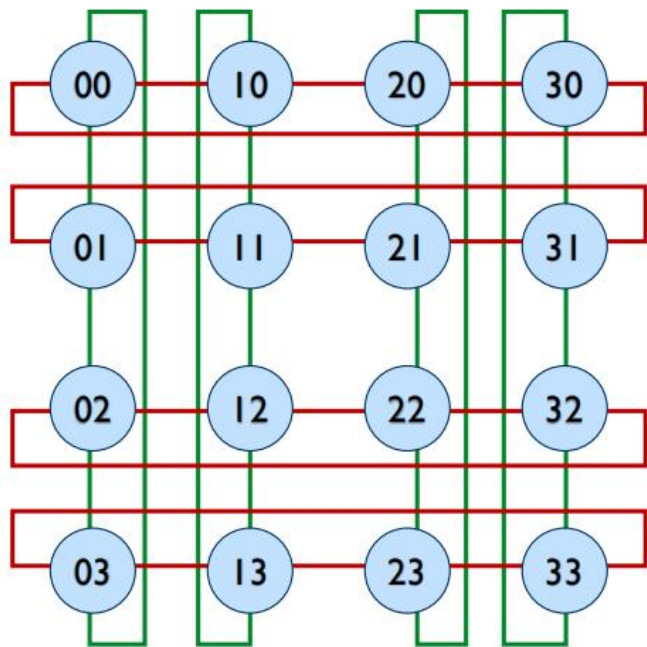


2D Torus

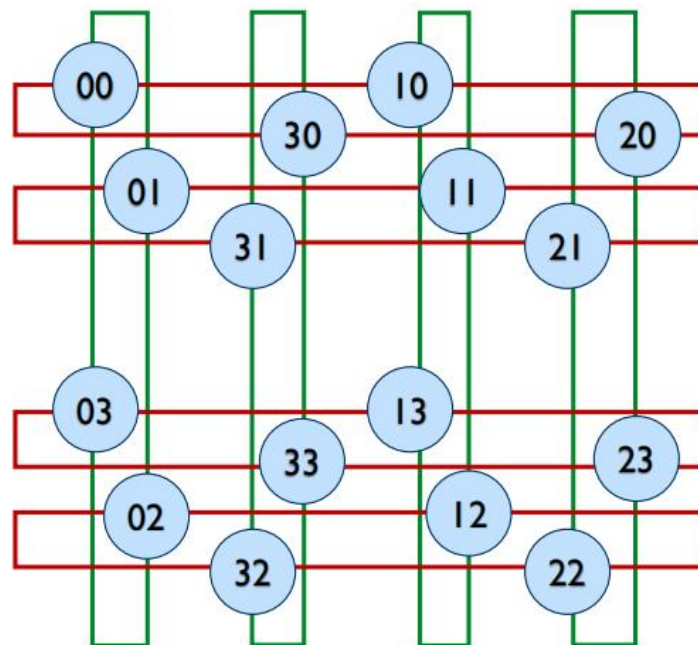
Torus (二维网络)

- 交错行和列可以消除对长连接的需求
- 所有连接的长度加倍

Torus

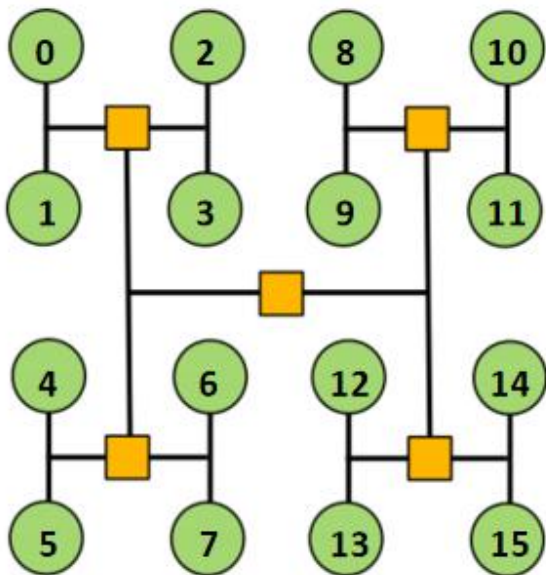


Folded Torus

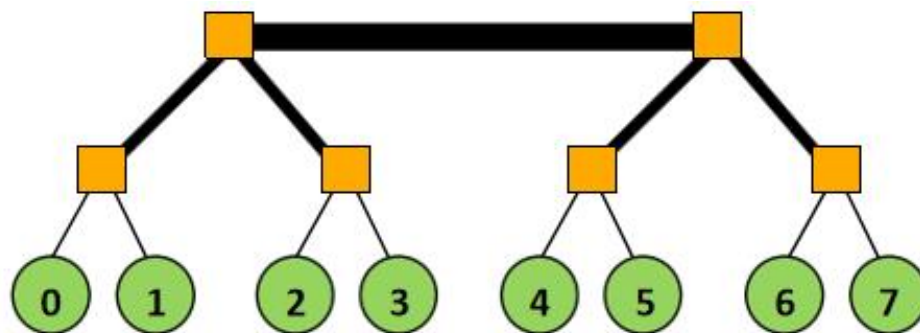


Tree (树)

- 平面, 层级式拓扑结构
- 类似mesh/torus, 当流量具有局部性时效果很好
- 延迟: $O(\lg N)$
- 使用 “胖树” 缓解根部节点带宽问题(根节点带宽需求高)



H-Tree



Fat Tree

Tree Routing

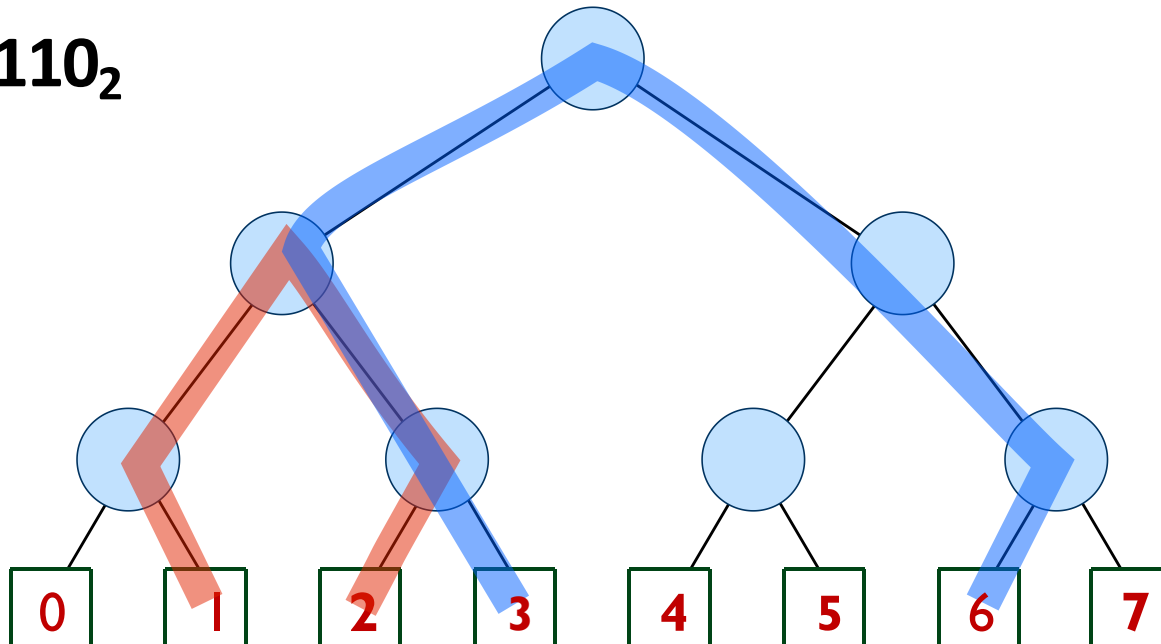
- Getting from 1 to 2:
 - $1 = 001_2$
 - $2 = 010_2$
- Getting from 3 to 6:
 - $3 = 011_2$
 - $6 = 110_2$

向上:

- 直到有共同祖先

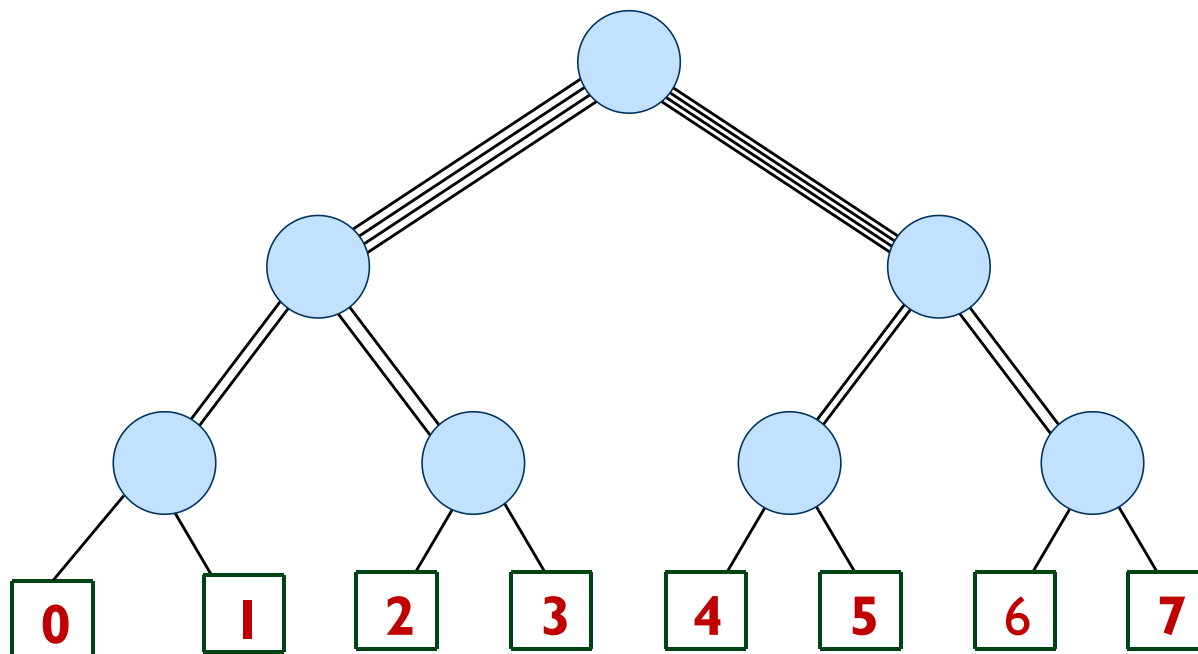
向下:

- 0 往左, 1 往右



Fat Tree(胖树)

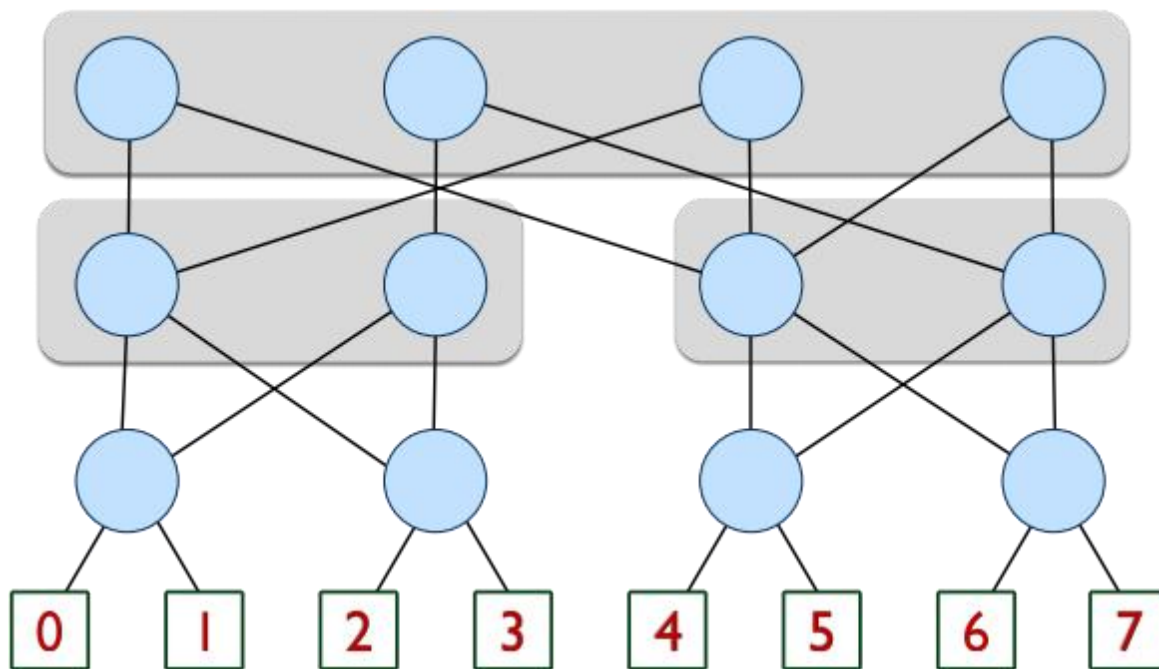
- 越上层节点带宽越高
- 分割带宽为 $O(N)$



类似于树形路由，但在多个链接可用时随机选择

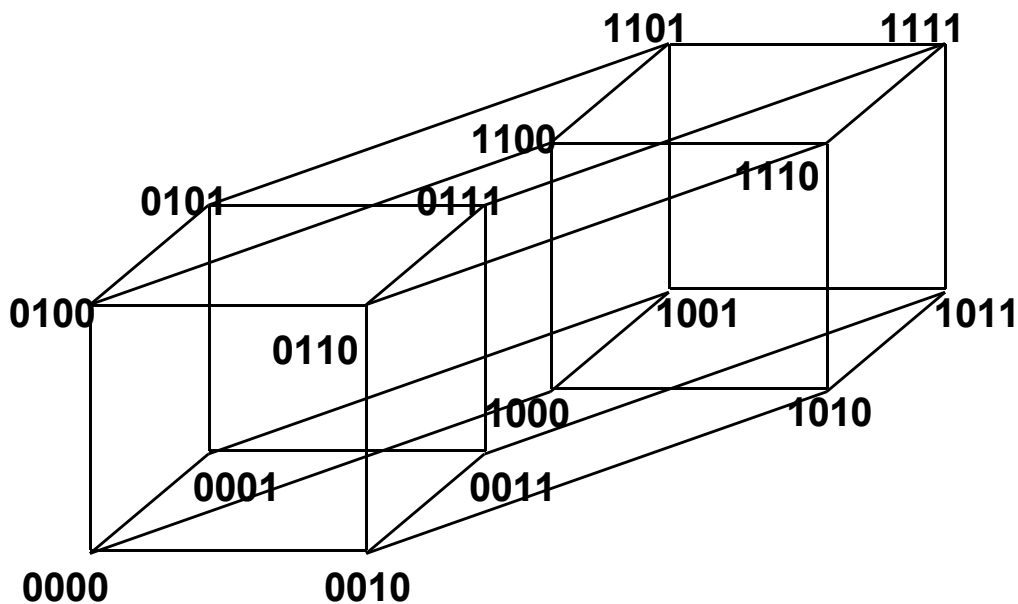
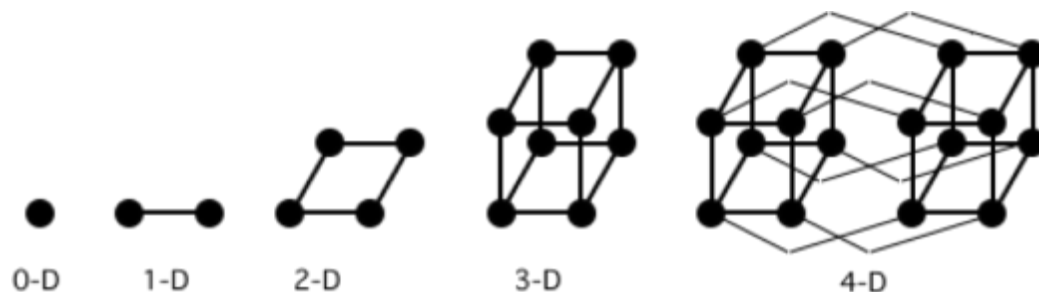
Constant-Width Fat Tree (等宽胖树)

- 有时称为“折叠的Clos网络”
- 每层节点数相同
- 常用于高性能集群网络



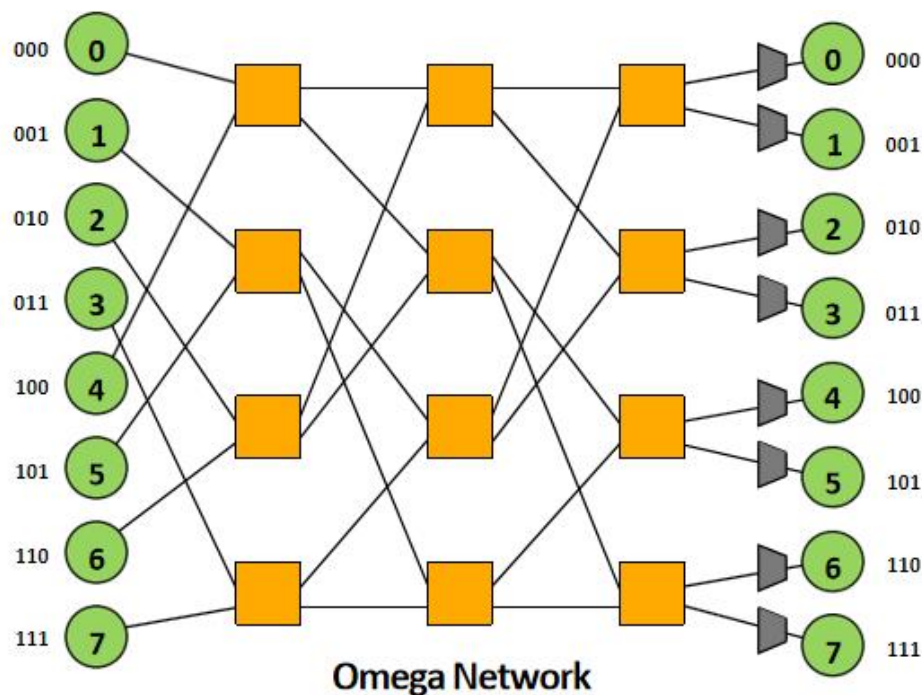
Hypercube(超立方)

- 低延迟: $O(\lg N)$
- 最大路径长度: $O(\lg N)$
- 连接数 $O(N \lg N)$



多级对数型网络

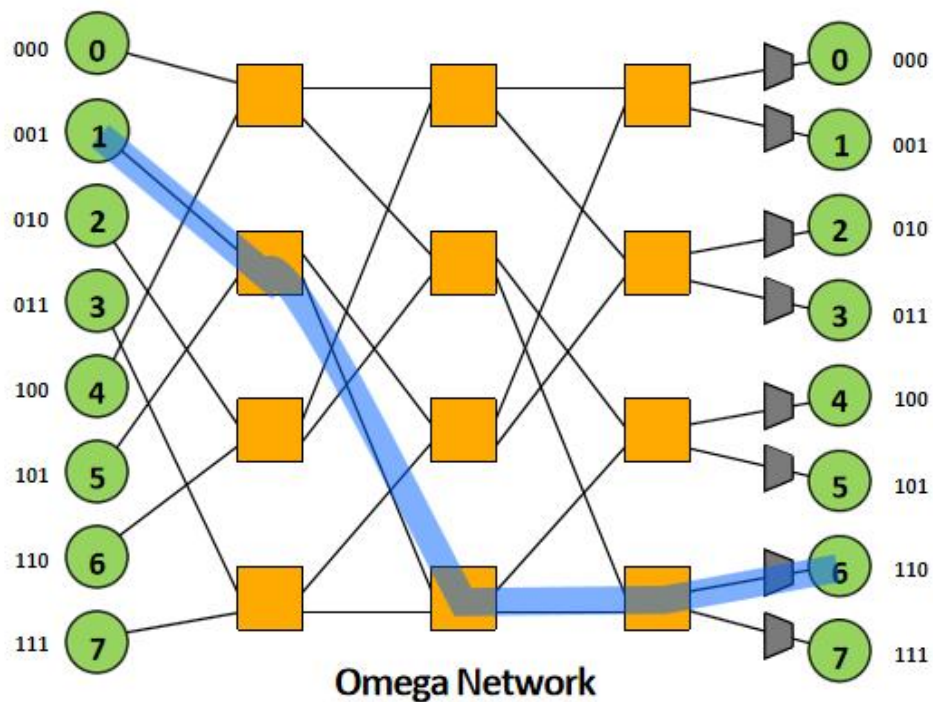
- 间接网络: 终端节点之间有多个交换机
- 成本: $O(N \lg N)$
- 延迟: $O(\lg N)$
- 多种拓补结构: Omega, butterfly, Clos networks, etc ...



多级对数型网络

- **Route from 1 to 6**

- $6 = 110_2$
- 1 $\rightarrow$ down, 0 $\rightarrow$ up



缓存和流量控制

(Buffering and flow control)

Circuit switching vs. packet switching

□ **Circuit switching**: 提前在发送和接收者之间建立一条完整链路用于发送信息

- 建立链路然后发送消息
- 高带宽
- 建立链路引入开销
- 资源利用率低



□ **Packet switching**: 为每个数据包做路由决策

- 每个包单独路由 (可能走不同链路)
- 资源利用率高
- 数据包动态转发带来开销
- 无建立/拆除链路开销



□ Message (消息)

- 网络节点(core, memory)之间的传输单元
- 可以通过多个数据包进行传输

□ Packet (数据包)

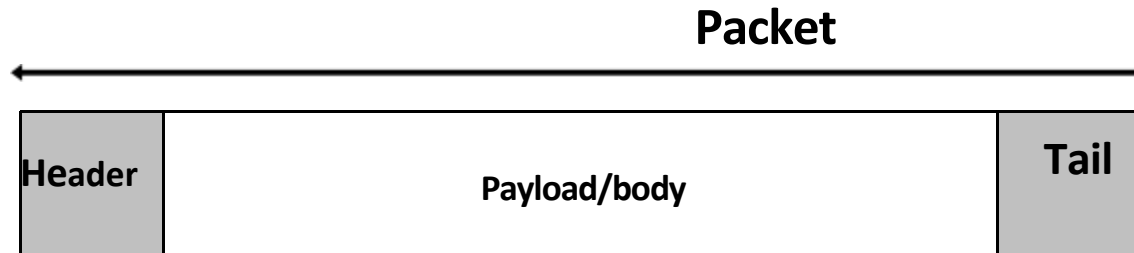
- 网络的传输单元
- 可以通过多个flits进行传输

□ Flit (flow control digit,流量控制单元)

- 数据包拆分成更小的单元，称为“flits”
- Flit: (“flow control digit”) 网络中的流量控制单元
- Flits是路由/缓存的最小粒度

数据包格式(Packet format)

- **Header:**
 - 包含路由和控制控制信息
 - 在数据包开始时，路由器可以提前开始转发
- **Payload/body:** 包含要发送的数据
- **Tail**
 - 包含控制信息, e.g., 纠错码
 - 通常位于数据包的末尾，以便可以边发送边生成



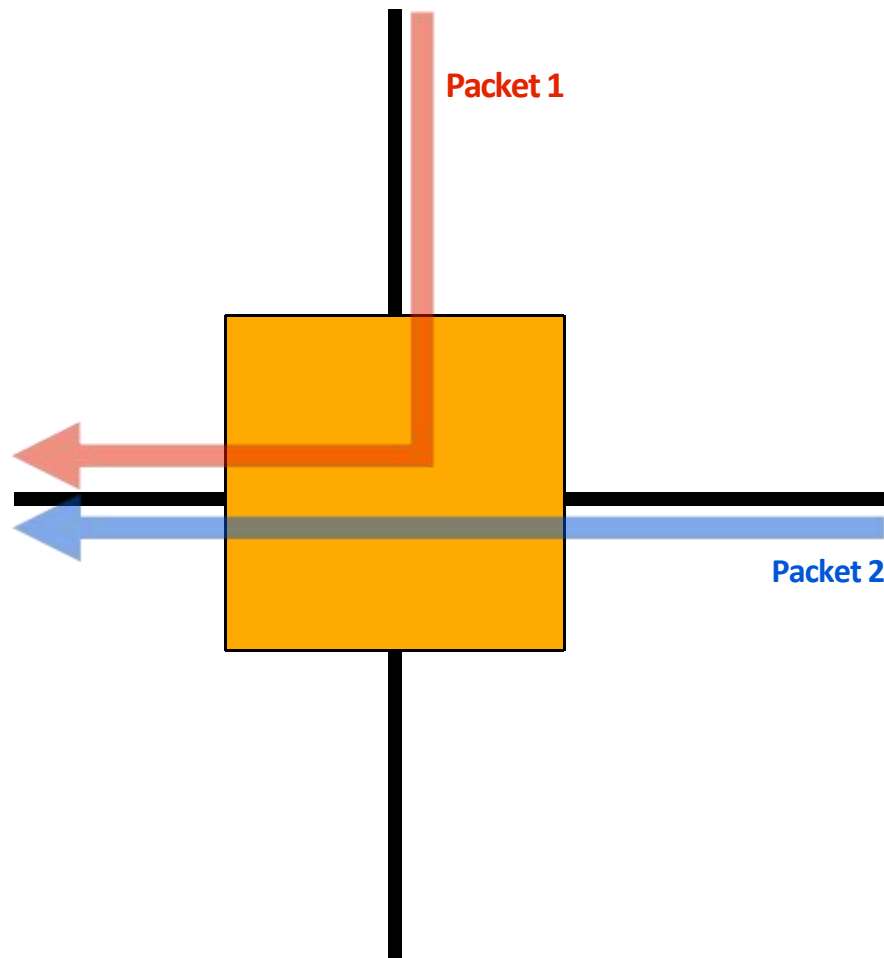
处理冲突

场景: 两个数据包需要同时路由到同一链路上

Options:

- ✓ 缓存一个数据包, 稍后发
- ✓ 丢掉一个数据包
- ✓ 重新路由一个数据包(偏转路由)

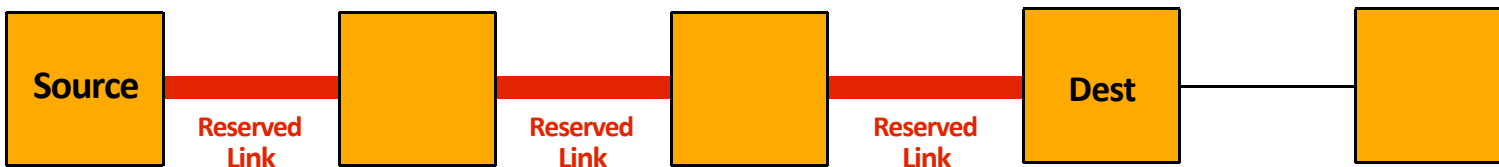
本课程只考虑缓存



Circuit-switched routing

□ 粗粒度资源分配

- 核心思想: 预先分配所有资源 (跨多个交换机的链路) 以用于发送消息



□ Costs

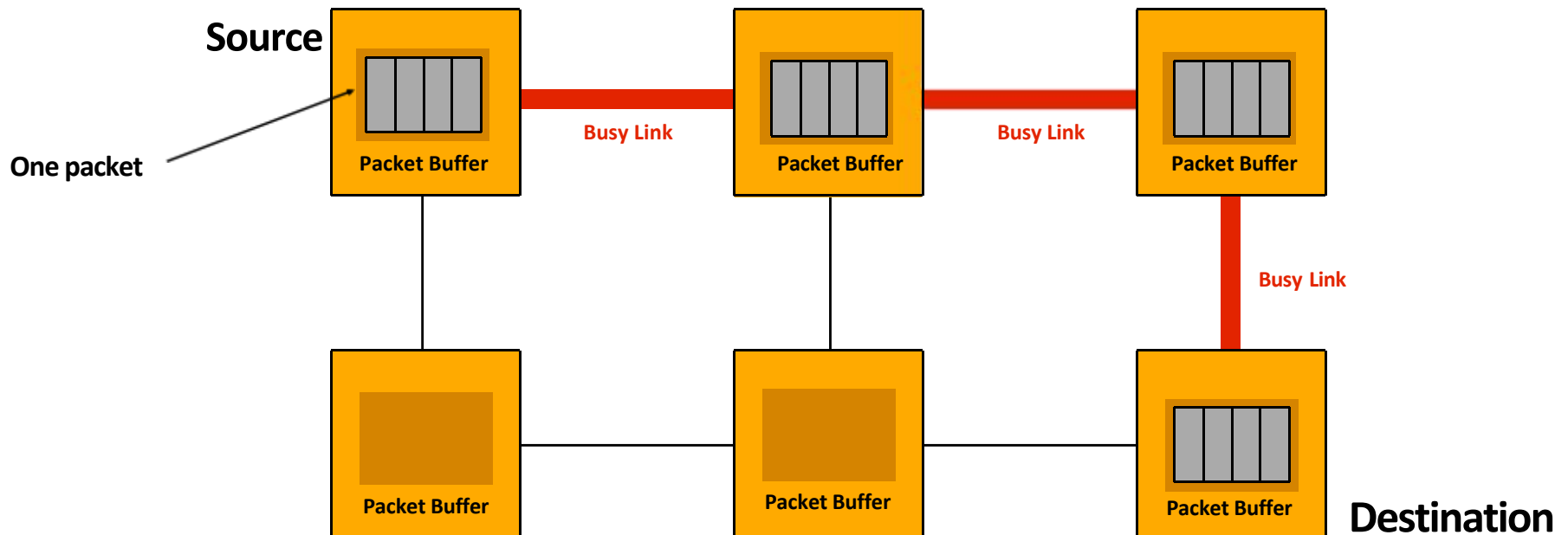
- 需要建立路径时间 (和拆除时间)
- 链路利用率低. 两个消息的传输不能共享相同的链路 (即使在传输过程中某些预分配路径上的资源不再被利用)

□ Benefits

- 由于预分配, 在传输过程中没有竞争, 因此无需缓存
- 任意消息大小 (路径一旦设置好, 就可以继续发送数据直到完成)

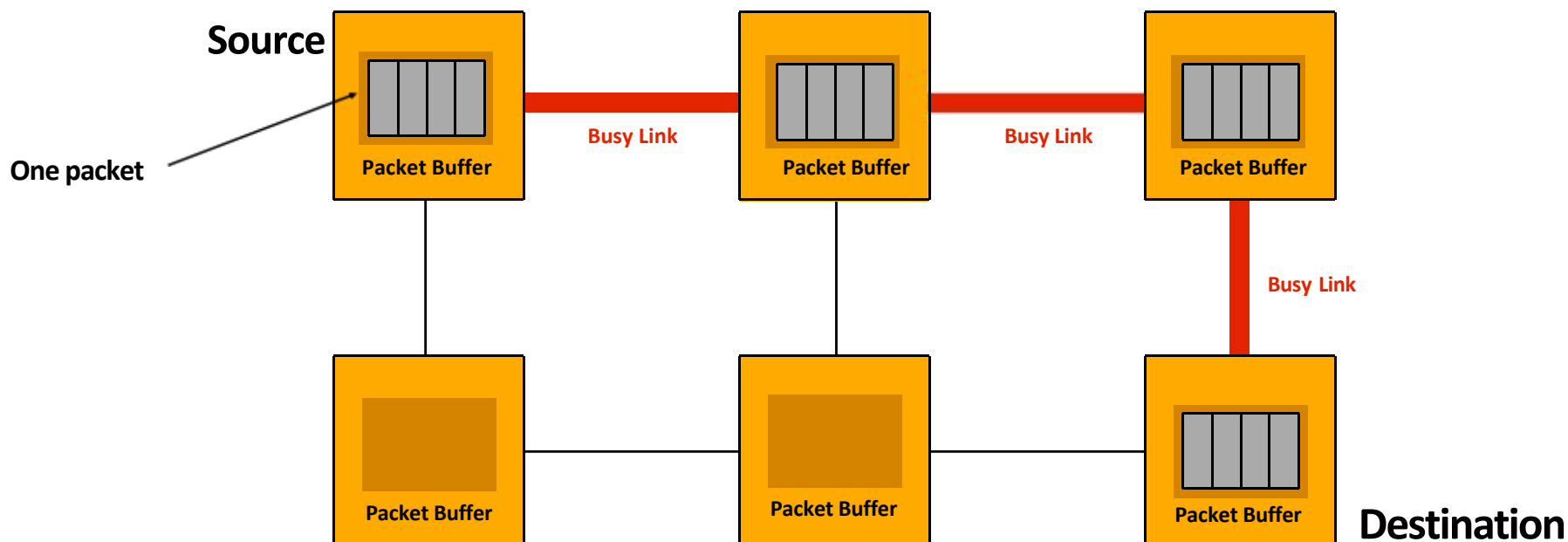
先存再转策略(Store-and-forward)

- 数据包在发往下一个节点之前，先整个拷贝到交换机
 - 同一个消息中不同的数据包可以走不同路径,但同一个数据包中的数据路径相同
- 需要在路由器中缓存整个数据包
- 每个数据包的传输延迟高



直通策略(Cut-through flow control)

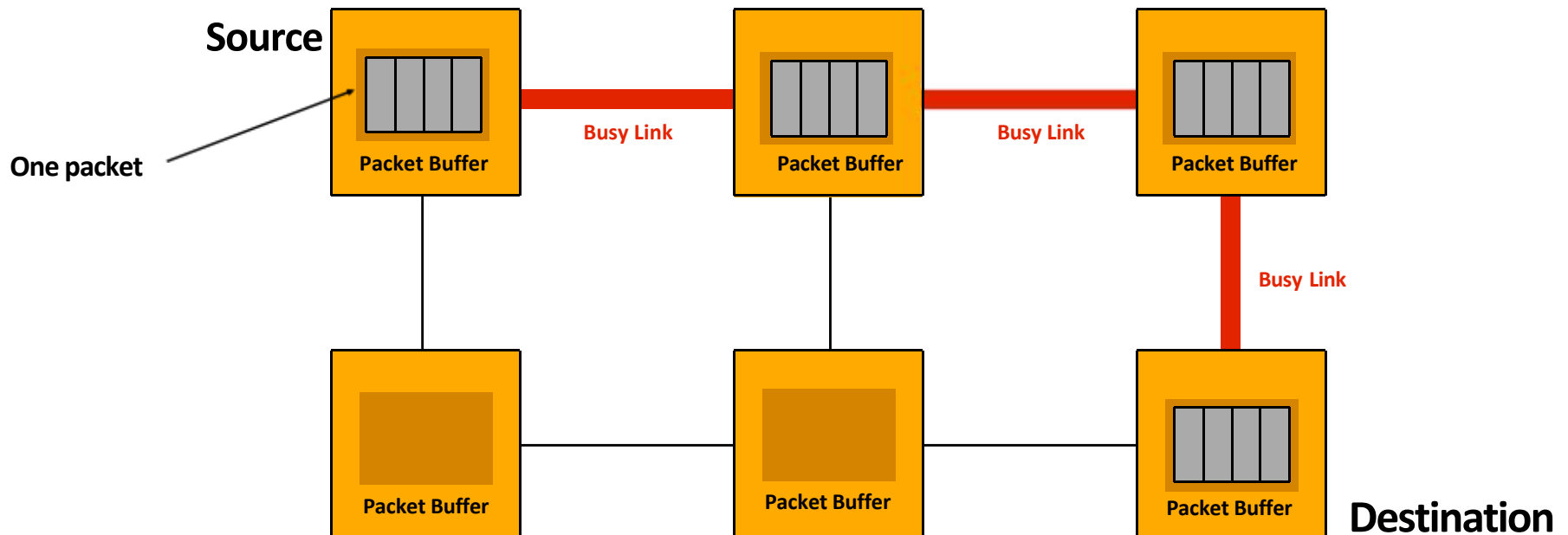
- **交换机一旦收到数据包头马上开始转发数据** (数据包头决定需要多少带宽以及转发到哪里)
- **减少传输延迟**
- **直通策略在高竞争情况下退化为先存再转**



直通策略(Cut-through flow control)

□ 如果输出链路被阻塞（无法传输头部），尾部的传输可以继续进行

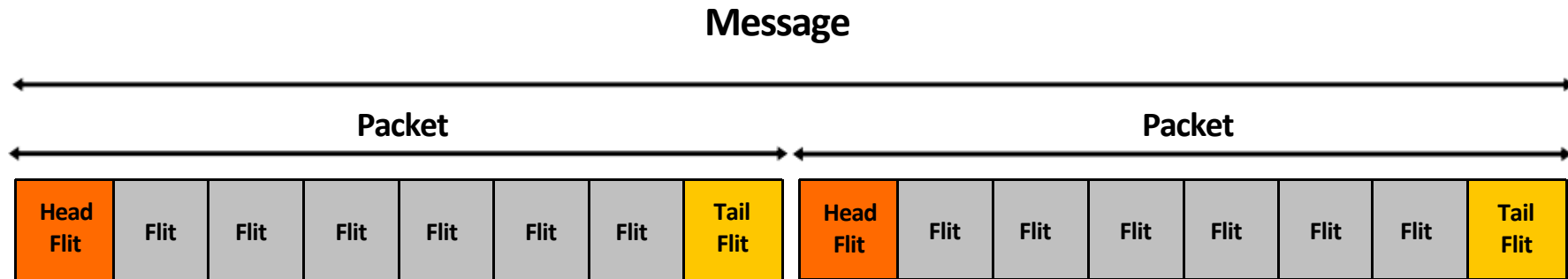
- **Worst case:** 整个消息被缓存在交换机 (退化为先存后转)



虫洞策略 (Wormhole flow control)

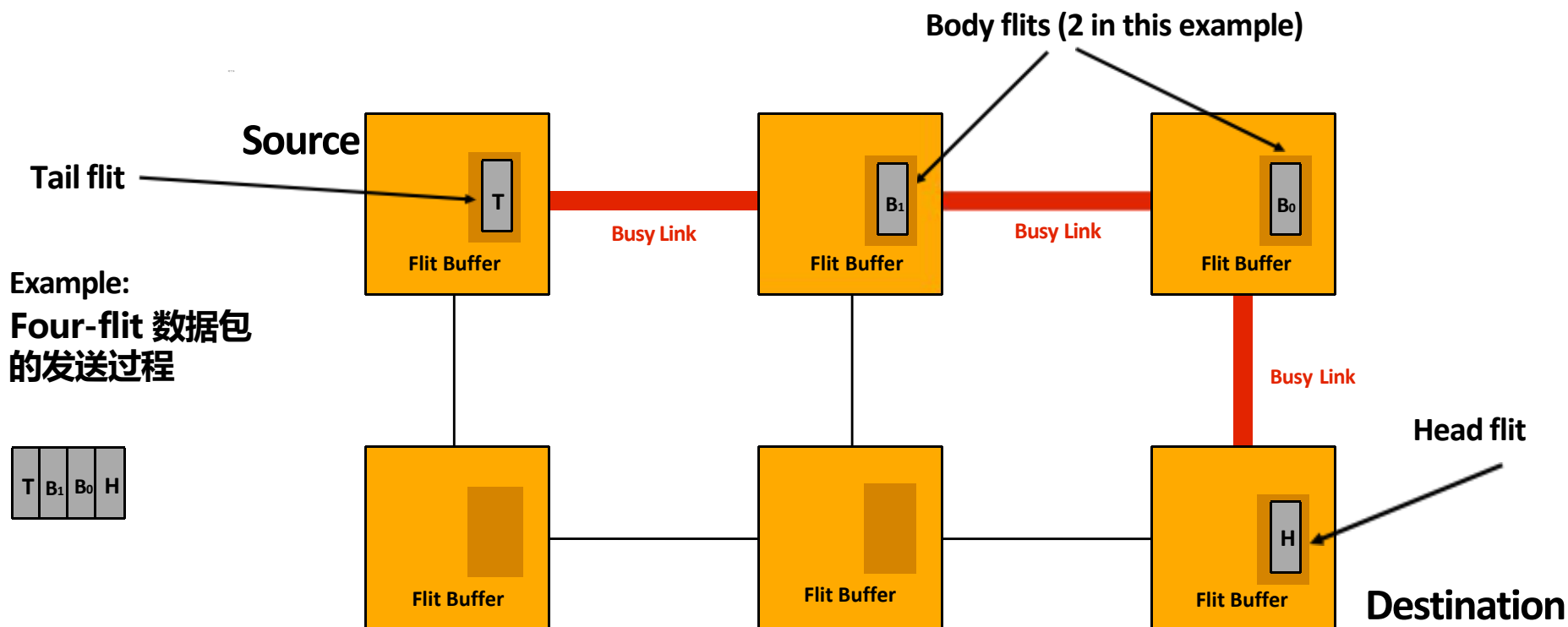
□ Flit (流量控制单元, flow control digit)

- 数据包拆分为更小的单元 “flits”
- Flits作为路由和缓存的最小粒度

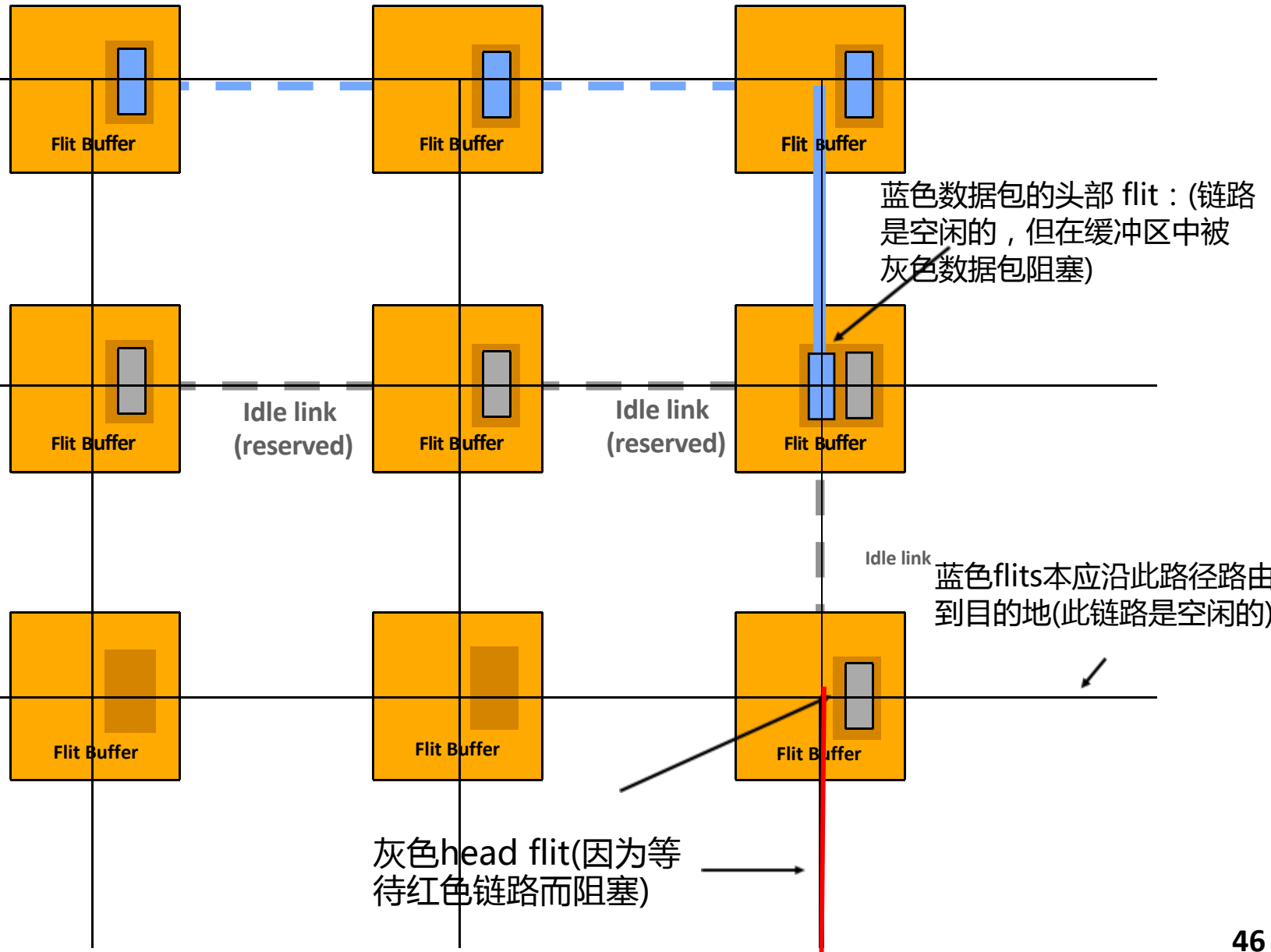


虫洞策略 (Wormhole flow control)

- 路由信息仅在head flit
- 中间的flits跟着head, 尾部flit跟着body
- 如果head flit阻塞, 剩余的部分也阻塞
- 完全流水线式传输

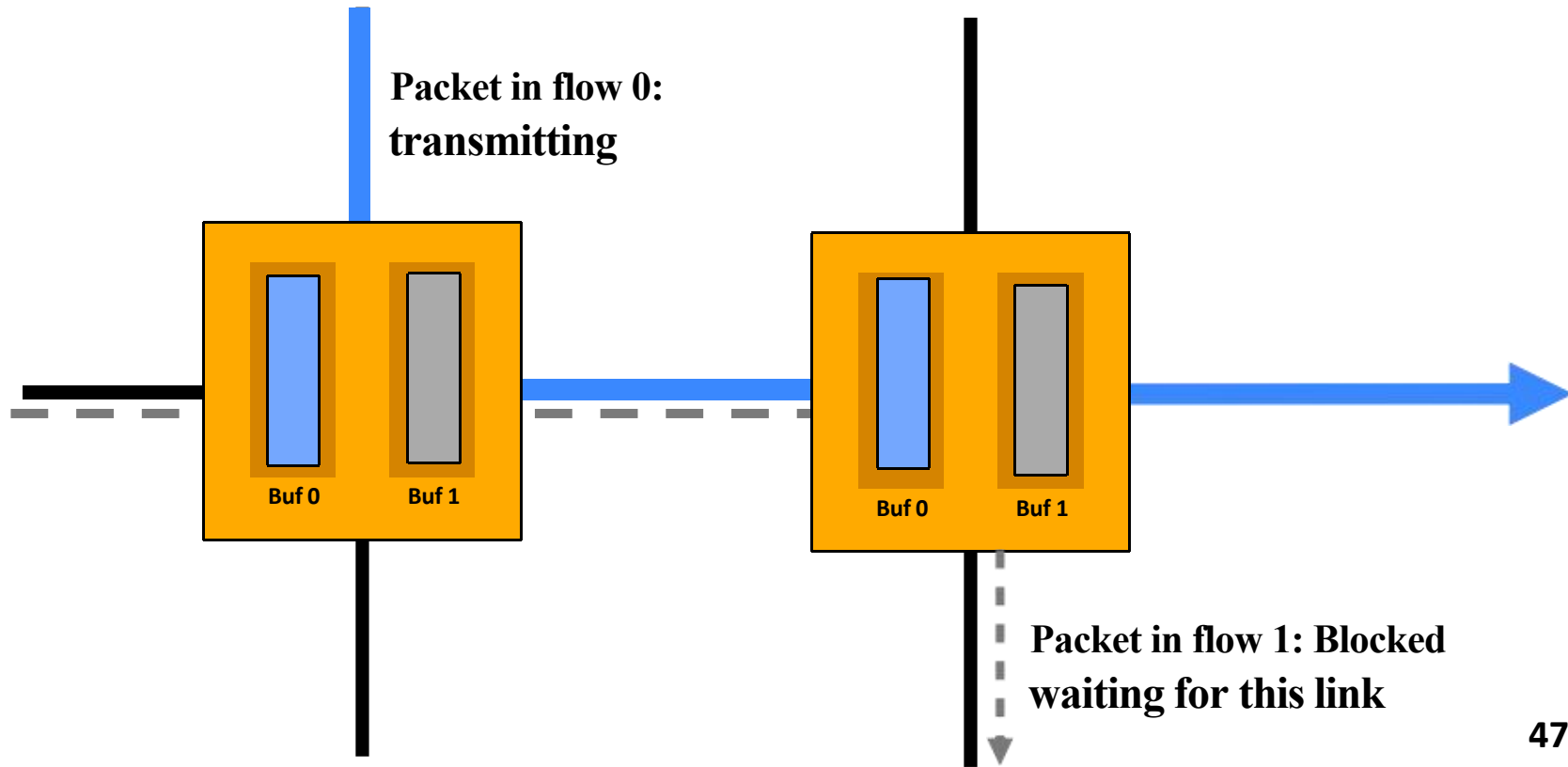


Problem: head-of-line blocking



虚拟通道 (Virtual channel flow control)

- 多个操作复用单一物理通道
- 将交换机的缓存空间分成多个缓存空间,共享同一个物理通道
- 减少head-of-line blocking



虚拟通道的其他用途

□ 避免死锁

- 可以打破资源的循环依赖
- 通过确保请求和响应使用不同的虚拟通道来防止循环
- 保留至少一个使用无死锁路由的虚拟通道

□ 流量类别的优先级划分

- 提供QoS保证
- 一些虚拟通道的优先级高于其他通道

□ 互联网络的能效问题

- 互联网络能耗很高 (~35% of total chip power in MIT RAW research processor)
- 无缓存网络
- 其他技术：开启/关闭网络的区域，使用快网络和慢网络

□ 优先级和QoS保证

- 按优先级处理数据包以提升多处理器性能（例如，一些应用程序可能对网络性能比其他应用程序更为敏感）
- 根据网络反馈调整端点（例如，处理器核心）的速度

□ 新兴技术

- 3D 堆叠
- 光子芯片网络（使用光波导代替电线）
- Reconfigurable devices (FPGAs): 创建针对特定应用的定制互连

□ 互联网络的性能对于整个多处理器性能非常关键

- 总线式连接扩展性差
- 历史上，互连是连接插槽、板卡、机架的外部网络
- 今天，所有这些问题都适用于芯片内网络的设计

□ 网络拓扑在性能、成本和复杂性上存在权衡差异

- e.g., crossbar, ring, mesh, torus, multi-stage network, fat tree, hypercube

□ 挑战: 高效路由策略

- 互连是一种宝贵的资源（通信成本高昂！）
- 基于 flit 的流量控制：细粒度流量控制以充分利用可用链路带宽
- 如果感兴趣，还有很多内容可以学习（在本课程中未讨论）：确保服务质量、优先级设置、可靠性、死锁、活锁等